2026s-CS758-Lecture14

Topics

- Learning with errors (LWE)
- Public-key encryption (PKE) from LWE

Sources

1. CS 294-168 Lattices, Learning with Errors and Post-Quantum Cryptography, Lecture 1, by Vaikuntanathan
2. A Decade of Lattice Cryptography, by Peikert

Plan

- Recall/Plan Today
  - In Module II, we
    - Introduced short-integer solution (SIS), the problem of finding a short vector in the (right) kernel of a random matrix $\bar{A} \in \mathbb{Z}_q^{n \times m}$. The problem can be considered to be (average-case) SVP in the dual q-ary lattice

$$\Lambda_q^{\perp}(\bar{A}) := \{\bar{e} \in \mathbb{Z}^m : \bar{A}\bar{e} = 0^n \bmod q\} \subseteq \mathbb{Z}^m.$$

    - Showed worst-case to average-case reduction for SIS
    - Constructed Minicrypt primitives like one-way function, collision-resistant hash function and commitment scheme using SIS
  - In Module III, we will
    - Introduce learning with errors (LWE), the problem of solving a system of noisy linear equations $\bar{x}^t \bar{A} \approx \bar{y} \bmod q$. Note that this is a different constraint compared to SIS. The problem can be considered to be CVP (BDD, to be precise) in the primal q-ary lattice

$$\Lambda_q(\bar{A}) := \{\bar{y} : \bar{y} \bmod q \in \bar{A}^t \mathbb{Z}_q^n\} \subseteq \mathbb{Z}^m.$$

    - Construct public-key encryption from LWE
      - PKE is a Cryptomania primitive, and implies Minicrypt primitives like OWF and commitment (see **Figure 1**)
    - Building on the PKE, construct *advanced* primitives like fully-homomorphic encryption (FHE) and identity-based encryption (IBE) from LWE
  - In today's lecture, we will
    - Define LWE, both its search and decision version, and study its relation with SIS
    - Use LWE to construct a symmetric-key encryption (SKE)

        - Build on the construction to obtain a PKE (Regev's scheme)
- Learning With Errors
    - *Definitions.*
        - **Definition 1 (Search LWE Problem).**
            - *Parameters:* $n, m, q \in \mathbb{N}$ (e.g., think of as in SIS) and an error distribution E over $\mathbb{Z}$, which outputs "small" values (see **Figure 2**), e.g.
                - *Discrete* Gaussian over $\mathbb{Z}$, centered at 0 and of width $s := \beta q$ for $\beta < 1$, i.e.,

$$D_s(x) := \rho_s(x)/\rho_s(\mathbb{Z})$$

                - Uniform distribution over $[-B, B]$ for some $B \ll q/2$
            - *Input:* $(\bar{A}, \bar{v}) \in \mathbb{Z}_q^{n \times m} \times \mathbb{Z}_q^m$, where
                - $\bar{A} \leftarrow \mathbb{Z}_q^{n \times m}$
                - $\bar{v}^t := \bar{x}^t \bar{A} + \bar{e}^t$ for $\bar{x} \leftarrow \mathbb{Z}_q^n$ and $\bar{e} \leftarrow E^m$, where E^m denotes m random and independent samples from E
            - *Solution:* $\bar{x}$
        - Note: the name "LWE" comes from the fact that the goal is to "learn" $\bar{x}$ given its noisy inner products with random vectors $\bar{a}_i$s
        - **Pop Quiz 1.**
            - *Q.* How can you view LWE as an instance of (average-case) CVP/BDD?
            - *A.* The lattice we consider is the primal q-ary lattice $\Lambda_q(\bar{A}) \subseteq \mathbb{Z}^m$, and the target point you are given is $\bar{v}$, which is obtained by perturbing the lattice point $\bar{x}^t \bar{A} \bmod q$ using error $\bar{e}^t$
        - Once can also define the decision problem corresponding to Definition 1 which, informally, requires distinguishing a World 0 in which you are given $(\bar{A}, \bar{v})$ as in Definition 1 from a World 1 in which you are given $(\bar{A}, \bar{v})$ with random $\bar{b}$ (see **Figure 3**).
        - **Definition 2 (Decision LWE Problem).**
            - *Input:*
                - Distribution 0: output $(\bar{A}, \bar{v})$ as in Definition 1
                - Distribution 1: output $(\bar{A}, \bar{v}) \leftarrow \mathbb{Z}_q^{n \times m} \times \mathbb{Z}_q^m$
            - *Decision:* Distribution 0 or 1?
        - The Search LWE (resp., Decision LWE) *assumption* states that Search LWE Problem (resp., Decision LWE Problem) is hard for PPT adversaries.
    - *Hardness of LWE.*
        - *Search vs. decision.* In Assignment 3, you will show that search and decision LWE are equivalent.
        - *LWE vs. SIS.*
            - **Claim 1.** Decision LWE is at most as hard as SIS.

- *Proof.* The reduction R, on input a Decision LWE instance $(\bar{A}, \bar{v})$ as in Definition 2, works as follows (see **Figure 4**):
  1. Call SIS solver on $\bar{A}$ to obtain short $\bar{s}$ such that $\bar{A}\bar{s} = 0^m \bmod q$
  2. Compute $\bar{v}^t \bar{s} \bmod q$
  3. If it is small, output 0; otherwise, output 1.
     To see why the reduction works, note that if $\bar{v} = \bar{x}^t \bar{A} + \bar{e}^t$ for some short $\bar{e}$, then

     $$\bar{v}^t \bar{s} = (\bar{x}^t \bar{A} + \bar{e}^t) \cdot \bar{s} = \bar{e}^t \bar{s} \bmod q,$$

     which is short whp. On the other hand, if $\bar{v}$ is random, then the above value is large whp.
- **Fact 2.** SIS is at most as *quantum-hard* as LWE.

- *Worst-case to average-case hardness.* Regev showed that given a (classical) solver for LWE, it is possible to construct a *quantum* solver for both GapSVP and SIVP (see **Figure 5**), stated formally in Theorem 1. Peikert later showed that given a (classical) solver for LWE, it is possible to construct a *classical* solver for GapSVP, but with caveats as stated in Theorem 2. (Note that the two results are incomparable.)
  - **Theorem 1 (Theorem 4.2.4 in [2]).** For any $m = \mathrm{poly}(n)$ and $2\sqrt{n}/\beta \le q \le 2^{\mathrm{poly}(n)}$, and discrete Gaussian of width $\beta q \ge 2\sqrt{n}$, solving Decision LWE (for parameters n, m, q and E set to Discrete Gaussian) is at least as hard as *quantumly* solving GapSVP$_\alpha$ and SIVP$_\alpha$ for n-dimensional lattices for $\alpha = \tilde{O}(n/\beta)$.
    - Here $\tilde{O}$ is the soft-O notation, which ignores polylogarithmic factors.
  - **Theorem 2 ([2]).** For any $m = \mathrm{poly}(n)$ and $2^{n/2} \le q \le 2^{\mathrm{poly}(n)}$, and discrete Gaussian of width $\beta q \ge 2\sqrt{n}$, solving Decision LWE (for parameters n, m, q and E set to Discrete Gaussian) is at least as hard as (classically) solving GapSVP$_\alpha$ for n-dimensional lattices for $\alpha = \tilde{O}(n/\beta)$.

- PKE from LWE
  - In this section, we will construct public-key encryption (PKE) from LWE. As a warm-up, we first construct symmetric-key encryption (SKE) from LWE. This construction highlights all the technical challenges with dealing with noise. For simplicity, we restrict to *bit* encryption -- extending to strings is not very difficult (see Exercise 3).
  - *Warm-up: SKE from LWE.*
    - Recall the definition of SKE:
      - **Definition 3.**
        - *Syntax:* a triple of algorithms $(\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$
          - $k \leftarrow \mathsf{KGen}(1^\lambda)$: the randomised *key-generation algorithm* KGen takes as input a security parameter λ in unary, and outputs a *shared* key k.

- $C \leftarrow \mathsf{Enc}_k(b)$: the randomised *encryption algorithm* Enc takes as input a message bit $b \in \{0,1\}$ and outputs a ciphertext C under k
- $b := \mathsf{Dec}_k(C)$: the deterministic *decryption algorithm* Dec takes as inputs a ciphertext C and a decryption key d, and outputs a message b

- *Requirements:*
  - *Correctness of decryption:* for every $\lambda \in \mathbb{N}$ and $b \in \{0,1\}$,

$$\Pr_{k \leftarrow \mathsf{KGen}(1^\lambda)}[\mathsf{Dec}_k(\mathsf{Enc}_k(b)) = 1.$$

  - *Ciphertext indistinguishability:* No efficient adversary A can distinguish ciphertexts of 0 and 1 i.e., for every PPT A,

$$\epsilon(\lambda) := \left| \Pr_{\substack{k \leftarrow \mathsf{KGen}(1^\lambda) \\ C \leftarrow \mathsf{Enc}_k(0)}}[\mathsf{A}(C) = 0] - \Pr_{\substack{k \leftarrow \mathsf{KGen}(1^\lambda) \\ C \leftarrow \mathsf{Enc}_k(1)}}[\mathsf{A}(C) = 0] \right|$$

    is negligible.

- *SKE from LWE.* Intuitively, we think of $\bar{x}$ as the shared key and use its noisy inner product with random $\bar{a} \leftarrow \mathbb{Z}_q^n$ as a *mask* to encrypt the bit b. Let us see exactly how (see **Figure 6**).
  - **Construction 1 (First attempt).**
    - $k \leftarrow \mathsf{KGen}(1^\lambda)$: sample $\bar{x} \leftarrow \mathbb{Z}_q^n$ (we will determine how to set n, m, q and E later) and output $k := \bar{x}$
    - $C \leftarrow \mathsf{Enc}_k(b)$: to encrypt $b \in \{0,1\}$, sample $\bar{a} \leftarrow \mathbb{Z}_q^n$, $e \leftarrow E$ and output $C := (C_1, C_2)$ where $C_1 := \bar{a}$ and

$$C_2 := \bar{x}^t \bar{a} + e + b \bmod q.$$

      - Note that C hides b thanks to Decision LWE assumption: the mask $\bar{x}^t \bar{a} + e$ is pseudo-random (even given $\bar{a}$) and thus acts as a (computational) one-time pad for b.
    - $b := \mathsf{Dec}_k(C =: (C_1, C_2))$: We can compute $C_2 - k^t C_1$ to recover $e + b$, but it is unclear how to extract b since e is random.
  - The idea to fix the issue with decryption in Construction 1 is to instead encrypt an "error-corrected" encoding of b (see **Figure 7**).
  - **Construction 2 ([1]).**
    - $k \leftarrow \mathsf{KGen}(1^\lambda)$: same as Construction 1
    - $C \leftarrow \mathsf{Enc}_k(b)$: to encrypt $b \in \{0,1\}$, sample $\bar{a} \leftarrow \mathbb{Z}_q^n$, $e \leftarrow E$ and output $C := (C_1, C_2)$ where $C_1 := \bar{a}$ and

$$C_2 := \bar{x}^t \bar{a} + e + \lfloor q/2 \rceil b \bmod q.$$

- $b := \mathsf{Dec}_k(C =: (C_1, C_2))$: Compute $C_2 - k^t C_1$ and output 0 iff its absolute value is less than $q/4$, i.e., output 0 iff

$$|C_2 - k^t C_1| < q/4.$$

- **Theorem 3.** Construction 2 is a secure SKE.
- *Proof.*
  - Correctness. Note that in an honestly-generated ciphertext $C := (C_1, C_2)$,

$$C_2 - k^t C_1 = e + \lfloor q/2 \rceil b \bmod q$$

    and thus as long as $e \in (-q/4, q/4)$ decryption will succeed since there is no overflow (see **Figure 7**). This is obviously true when E is uniform over $(-B, B)$ for constant $B \ll q/4$.
    - **Exercise 1.** Show that the above holds whp when E is discrete Gaussian. How do you set βq for the argument to work?
  - Ciphertext indistinguishability more or less follows from decision LWE assumption.
    - **Exercise 2.** Formally prove ciphertext indistinguishability.
- **Exercise 3 (Boosting Construction 2).**
  1. Prove that Construction 2 satisfies CPA security.
  2. Extend Construction 2 to multiple bits and prove its CPA security.

- *Extending to PKE (Regev's PKE).*
  - Let's first recall the definition of PKE:
    - **Definition 4.**
      - *Syntax:* a triple of algorithms $(\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$
        - $(ek, dk) \leftarrow \mathsf{KGen}(1^\lambda)$: the randomised *key-generation algorithm* KGen takes as input a security parameter λ in unary, and outputs a public encryption key ek and a private decryption key dk.
        - $C \leftarrow \mathsf{Enc}_e(b)$: the randomised *encryption algorithm* Enc takes as input a message bit $b \in \{0, 1\}$ and outputs a ciphertext C under ek
        - $b := \mathsf{Dec}_{dk}(C)$: the deterministic *decryption algorithm* Dec takes as inputs a ciphertext C and a decryption key dk, and outputs a bit b
      - *Requirements:*
        - *Correctness of decryption:* for every λ and b,

$$\Pr_{(ek,dk) \leftarrow \mathsf{KGen}(1^\lambda)}[\mathsf{Dec}_{dk}(\mathsf{Enc}_{ek}(b)) = 1.$$

- *Ciphertext indistinguishability:* No PPT adversary A can distinguish ciphertexts of 0 and 1 i.e., for every efficient A ,

$$\epsilon(\lambda) := \left| \Pr_{\substack{(ek,dk)\leftarrow\mathsf{KGen}(1^\lambda)\\ C\leftarrow\mathsf{Enc}_{ek}(0)}}[\mathsf{A}(ek,C)=0] - \Pr_{\substack{(ek,dk)\leftarrow\mathsf{KGen}(1^\lambda)\\ C\leftarrow\mathsf{Enc}_{ek}(1)}}[\mathsf{A}(ek,C)=0] \right|$$

  is negligible.

- The construction is similar to Construction 2. The main idea is to set an LWE instance $(\bar{A},\bar{v})$ as the public key with the underlying solution $\bar{x}$ as the secret key, and use a *random subset* of these instances to generate the mask for encrypting.
- **Construction 3 ("Regev PKE"[1]).**
  - $(ek,dk)\leftarrow\mathsf{KGen}(1^\lambda)$: generate $(\bar{A},\bar{v}^t := \bar{x}^t\bar{A}+\bar{e}^t)$ as in Definition 1 (with $m\geq n\lfloor\log(q)\rceil$) and output $ek := (\bar{A},\bar{v})$ and $dk := \bar{x}$.
  - $C\leftarrow\mathsf{Enc}_{ek}(b)$: to encrypt $b\in\{0,1\}$, sample $\bar{r}\leftarrow\{0,1\}^m$ and output $C := (C_1,C_2)$ where $C_1 := \bar{A}\bar{r}$ and

$$C_2 := \bar{v}^t\bar{r} + \lfloor q/2\rceil b \bmod q.$$

  - $b := \mathsf{Dec}_{dk}(C =: (C_1,C_2))$: Compute $C_2 - dk^tC_1$ and output 0 iff its absolute value is less than $q/4$, i.e., output 0 iff

$$|C_2 - dk^tC_1| < q/4.$$

  - **Theorem 4.** Construction 3 is a secure PKE.
  - *Proof.*
    - Correctness. Observe that

$$C_2 - dk^tC_1 = \bar{e}^t\bar{r} + \lfloor q/2\rceil b \bmod q.$$

      and thus decryption succeeds as long as $\bar{e}^t\bar{r}\in(-q/4,q/4)$. This, in turn, holds as long as each of $\bar{e}$'s component

$$e_i \in (-q/4(m+1), q/4(m+1))$$

      and thus we can set $B \ll q/4(m+1)$.
      - **Exercise 4.** Show that the above holds whp when E is discrete Gaussian. How do you set βq for the argument to work?
    - Ciphertext indistinguishability can be argued by a hybrid argument where we step-by-step replace the encryption key $ek := (\bar{A},\bar{v})$ and ciphertertext $C := (C_1,C_2)$ with random values:
      1. H_0: Generate ek and C as in Construction 3.
      2. H_1: Replace the $\bar{v}$ in the ek with a random value, i.e., $\bar{v}\leftarrow\mathbb{Z}_q^m$. This switch is indistinguishable thanks to decision LWE assumption.

3. H_2: Replace C_2 with a random value, i.e., $C_2 \leftarrow \mathbb{Z}_q$. This switch is *statistically* indistinguishable by Leftover Hash Lemma, which in our context states that

$$\mathsf{SD}((\bar{A}, \bar{A}\bar{r}), (\bar{A}, \bar{u})) = \mathrm{negl}(\lambda)$$

where $\bar{u} \leftarrow \mathbb{Z}_q^n$. At this point, C contains no information about the bit b and thus security is guaranteed information theoretically.

Further Reading

-

2026s-CS758-Lecture15

Topics

- Public-key encryption (PKE) from LWE: Regev PKE and its variants
- Additively-homomorphic PKE

Sources

1. CS 294-168 Lattices, Learning with Errors and Post-Quantum Cryptography, Lecture 1, by Vaikuntanathan
2. CS 294-168 Lattices, Learning with Errors and Post-Quantum Cryptography, Lecture 6, by Vaikuntanathan

Plan

- Recall/Plan Today
  - In the last lecture:
    - We first introduced the LWE problem, the problem of solving a system of noisy linear equations $\bar{v}^t \approx \bar{x}^t \bar{A}$. Alternatively, it can be thought of as average-case bounded-distance decoding (BDD) problem on the primal q-ary lattice defined by $\bar{A}$. We saw that its decision and search variants are equivalent and -- like SIS -- it enjoys worst-case to average-case hardness guarantees.
    - Then we used LWE to construct an SKE. Recall that in our construction Alice and Bob shared a random vector $\bar{x} \in \mathbb{Z}_q^n$ as the key and the idea (for Alice or Bob) to encrypt a bit b was to *mask* (an appropriate encoding of) it with a noisy random inner product with $\bar{x}$: i.e., the ciphertext was

      $$c := (c_1, c_2) = (\bar{a}, \bar{x}^t \bar{a} + e + \lfloor q/2 \rceil b \bmod q)$$

      for $\bar{a} \leftarrow \mathbb{Z}_q^n$ and $e \leftarrow E$ (see **Figure 1**). To decrypt, we *unmasked* and then checked whether the result is closer to 0 or $q/2$: i.e., output 0 iff

      $$|c_2 - k^t c_1 \bmod q| < q/4.$$

      We then argued that as long as the error is not too large, e.g., when use set E to be uniform over $(-B, B)$ for $B \ll q/4$, decryption is guaranteed to work.
    - In fact, the takeaway from last lecture was dealing with noise.
  - In today's lecture:
    - We will build on our SKE to construct public-key encryption (PKE) from LWE.
    - First, we will describe Regev PKE and then discuss a couple of variants.
    - Finally, we will see that Regev PKE (and its variants) enjoy a nice property in additive homomorphism, i.e., "adding" two ciphertexts leads to a ciphertext of the added messages.

- PKE from LWE: Regev PKE
  - Let's first recall the definition of PKE:
    - **Definition 1 (PKE).**
      - *Syntax:* a triple of algorithms $(\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$
        - $(ek, dk) \leftarrow \mathsf{KGen}(1^\lambda)$: the randomised *key-generation algorithm* KGen takes as input a security parameter λ in unary, and outputs a public encryption key ek and a private decryption key dk.
        - $c \leftarrow \mathsf{Enc}_e(b)$: the randomised *encryption algorithm* Enc takes as input a message bit $b \in \{0,1\}$ and outputs a ciphertext c under ek
        - $b := \mathsf{Dec}_{dk}(c)$: the deterministic *decryption algorithm* Dec takes as inputs a ciphertext c and a decryption key dk, and outputs a bit b
      - *Requirements:*
        - *Correctness of decryption:* for every λ and b,

$$\Pr_{(ek,dk)\leftarrow \mathsf{KGen}(1^\lambda)}[\mathsf{Dec}_{dk}(\mathsf{Enc}_{ek}(b) = b) = 1.$$

        - *Ciphertext indistinguishability:* No efficient adversary A can distinguish ciphertexts of 0 and 1 i.e., for every PPT A ,

$$\epsilon(\lambda) := \left| \Pr_{\substack{(ek,dk)\leftarrow \mathsf{KGen}(1^\lambda) \\ c\leftarrow \mathsf{Enc}_{ek}(0)}}[\mathsf{A}(ek, c) = 0] - \Pr_{\substack{(ek,dk)\leftarrow \mathsf{KGen}(1^\lambda) \\ c\leftarrow \mathsf{Enc}_{ek}(1)}}[\mathsf{A}(ek, c) = 0] \right|$$

          is negligible.
  - *Overview of Regev PKE.* The PKE construction is similar to the SKE construction from Lecture 14 (Construction 2). In that construction, Alice and Bob shared a secret $\bar{x}$ and this allowed Alice or Bob to generate random noisy inner products with $\bar{x}$, which served as the mask. The immediate question that arises is how do we mimic this in the public-key setting since there is no shared key? The main idea in Regev PKE is for Alice to set an LWE instance $(\bar{A}, \bar{v})$ *as the public encryption key* with the underlying solution $\bar{x}$ acting as the secret decryption key. What Bob can now do is use a *random subset* of these instances to generate the mask for encryption! Let us see the construction formally.
  - **Construction 1 ("Regev PKE"[1]).**
    - $(ek, dk) \leftarrow \mathsf{KGen}(1^\lambda)$: generate $(\bar{A}, \bar{v}^t := \bar{x}^t\bar{A} + \bar{e}^t)$ as in Lecture 14, Definition 1 with $m \geq 2(n+1)\lceil \log(q) \rceil$ and output $ek := (\bar{A}, \bar{v})$ and $dk := \bar{x}$.
    - $c \leftarrow \mathsf{Enc}_{ek}(b)$: to encrypt $b \in \{0,1\}$, sample $\bar{r} \leftarrow \{0,1\}^m$ and output

$$c := (c_1, c_2) := (\bar{A}\bar{r}, \bar{v}^t\bar{r} + \lfloor q/2 \rceil b) \bmod q.$$

    - $b := \mathsf{Dec}_{dk}(c =: (c_1, c_2))$: Compute $c_2 - dk^t c_1$ and output 0 iff its absolute value is less than $q/4$, i.e., output 0 iff

$$|c_2 - dk^t c_1 \bmod q| < q/4.$$

- **Theorem 1.** Construction 1 is a secure PKE.
- *Proof.*
  - *Correctness.* Observe that

    $$c_2 - dk^t c_1 = \bar{e}^t \bar{r} + \lfloor q/2 \rceil b \bmod q.$$

    and thus decryption succeeds as long as $\bar{e}^t \bar{r} \in (-q/4, q/4)$. This, in turn, holds as long as each of $\bar{e}$'s component

    $$e_i \in (-q/4(m+1), q/4(m+1))$$

    and thus we can set $B \ll q/4(m+1)$.
    - **Exercise 1.** We say that a PKE has *decryption error* $\delta(\lambda)$ if the probability with which correctness holds in Definition 1 is at least $1 - \delta(\lambda)$. Show that the decryption error of Construction 1 is negligible when E is discrete Gaussian. How do you set βq for the argument to work?
  - *Ciphertext indistinguishability.* We proceed by a hybrid argument in which the encryption key ek and ciphertext c are step-by-step replaced with random values:
    1. H_0: Generate ek and C as in Construction 1. That is, the H_0 distribution is

       $$((\bar{A}, \bar{v} := \bar{s}^t \bar{A} + \bar{e}), (\bar{A}\bar{r}, \bar{v}^t \bar{r} + \lfloor q/2 \rceil b)),$$

       with $\bar{A}$, $\bar{s}$, $\bar{e}$ and $\bar{r}$ are sampled as in Construction 1, and all operations are modulo q (which we henceforth omit).
    2. H_1: Replace the $\bar{v}$ in the ek with a random value, i.e., $\bar{v} \leftarrow \mathbb{Z}_q^m$, i.e., the H_1 distribution is

       $$((\bar{A}, \bar{v}), (\bar{A}\bar{r}, \bar{v}^t \bar{r} + \lfloor q/2 \rceil b)).$$

       This switch is indistinguishable thanks to decision LWE assumption.
       - **Exercise 2.** Prove this claim.
    3. H_2: Replace c with a random $(c_1, c_2) \leftarrow \mathbb{Z}_q^{n+1}$ resulting in the distribution

       $$((\bar{A}, \bar{v}), (c_1, c_2)).$$

       This switch can be shown to be *statistically* indistinguishable using the so-called Leftover Hash Lemma (LHL), stated formally below (for our context) using the notion of min-entropy of a distribution P, which informally is the maximum length of encoding of an element in the support of P using Huffman coding.
       - **Lemma 1 (Lemma 23 in [2]).** Let P be a probability distribution over $\mathbb{Z}^m$, and let $H_\infty(P)$ denote its min-entropy, i.e.,

         $$H_\infty(P) := -\log \max_{\bar{v} \in \mathbb{Z}^m} P(\bar{v}).$$

Moreover, let $\bar{A}' \leftarrow \mathbb{Z}_q^{(n+1)\times m}$ and $c \leftarrow \mathbb{Z}_q^{n+1}$. If $H_\infty(P) \geq (n+1)\log(q) + 2\log(1/\varepsilon)$ then for $\bar{r} \leftarrow P$ we have

$$\mathsf{SD}((\bar{A}', \bar{A}'\bar{r}), (\bar{A}', c)) \leq \varepsilon.$$

- In our context,

$$\bar{A}' := \begin{pmatrix} \bar{A} \\ \bar{v}^t \end{pmatrix} \leftarrow \mathbb{Z}_q^{(n+1)\times m} \text{ and } c := \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} \leftarrow \mathbb{Z}_q^{n+1},$$

and we have

$$\mathsf{SD}((\bar{A}', \bar{A}'\bar{r}), (\bar{A}', c) = \mathrm{negl}(\lambda)$$

as we argue next. Since $\bar{r} \leftarrow \{0,1\}^m$, $H_\infty(P) = m$. Moreover, since we set $m \geq 2(n+1)\lceil\log(q)\rceil$, we have

$$2\log(1/\varepsilon) \leq (n+1)\lceil\log(q)\rceil \approx O(\lambda) \Leftrightarrow 1/\varepsilon \leq 2^{O(\lambda)} \Leftrightarrow \varepsilon \geq 2^{-O(\lambda)}.$$

4. At this point, c contains no information about the bit b. Security is thus guaranteed information-theoretically.

- **Exercise 1 (Boosting Construction 1).**
  1. Prove that Construction 1 satisfies CPA security.
  2. Extend Construction 1 to multiple bits and prove its CPA security.

- Variants of Regev PKE
  1. "Dual Regev" PKE
     - Regev PKE has the following properties:
       1. The public key is *not* uniformly random due to $\bar{v}$ being sampled according to LWE distribution
       2. The secret key $\bar{x}$ corresponding to a public key $(\bar{A}, \bar{v})$ is *unique* (since $\bar{e}$ is chosen so that it has norm at most minimum distance)
       3. Different random coins may yield the same ciphertext since there are collisions in the map $\bar{r} \mapsto \bar{A}\bar{r}$
     - In "Dual Regev" PKE the *roles of secret key and random coins* are swapped and thus it ends up having the following properties:
       1. The public key *is uniform* -- we will exploit this to construct IBE later on
       2. There are many valid secret keys corresponding to a public key
       3. A ciphertext corresponds to a *unique random coin*
     - The scheme is described below.
       - **Construction 2 ("Dual Regev" PKE [1]).**
         - $(ek, dk) \leftarrow \mathsf{KGen}(1^\lambda)$: Sample $\bar{A} \leftarrow \mathbb{Z}_q^{n\times m}$ with $m \geq 2n\lceil\log(q)\rceil$ and $\bar{r} \leftarrow \{0,1\}^m$. Output $ek := (\bar{A}, \bar{v} := \bar{A}\bar{r})$ and $dk := \bar{r}$.
         - $c \leftarrow \mathsf{Enc}_{ek}(b)$: to encrypt $b \in \{0,1\}$, sample $\bar{x} \leftarrow \mathbb{Z}_q^n$, $\bar{e}, e \leftarrow E$ and output $c := (c_1, c_2)$ where

$$c_1 := \bar{x}^t \bar{A} + \bar{e}$$

and

$$c_2 := \bar{x}^t \bar{v} + e + \lfloor q/2 \rceil b \bmod q.$$

- $b := \mathsf{Dec}_{dk}(c =: (c_1, c_2))$: Compute $c_2 - dk^t c_1$ and output 0 iff its absolute value is less than $q/4$, i.e., output 0 iff

$$|c_2 - dk^t c_1 \bmod q| < q/4.$$

- **Exercise 2.** Show that Construction 2 is a secure PKE. (*Hint:* the proof is similar to that for Construction 1, but with certain steps reversed)

2. "LPR PKE".
   - Recall that in Regev and Dual Regev PKEs, the secret keys are sampled as $\bar{x} \leftarrow \mathbb{Z}_q^n$ and $\bar{r} \leftarrow \{0,1\}^m$ respectively.
   - **Pop Quiz 1.**
     - *Q.* How much random coins are required to sample these?
     - *A.* At least $m \approx n\lceil \log(q) \rceil$ bits
   - LPR PKE samples secret key *from the error distribution* which might be possible using fewer random coins. Additionally, this switch allows shorter public keys since we will not rely on LHL. The construction is described below.
     - **Construction 3 (LPR PKE [1]).**
       - $(ek, dk) \leftarrow \mathsf{KGen}(1^\lambda)$: sample $\bar{A} \leftarrow \mathbb{Z}_q^{n \times n}$, $\bar{x}, \bar{e} \leftarrow E^n$ and output $ek := (\bar{A}, \bar{v}^t := \bar{x}^t \bar{A} + \bar{e}^t)$ and $dk := \bar{x}$.
       - $c \leftarrow \mathsf{Enc}_{ek}(b)$: to encrypt $b \in \{0,1\}$, sample $\bar{r}, \bar{d}, e \leftarrow E$ and output $c := (c_1, c_2)$ where

$$c_1 := \bar{A}\bar{r} + \bar{d}$$

and

$$c_2 := \bar{v}^t \bar{r} + e + \lfloor q/2 \rceil b \bmod q.$$

       - $b := \mathsf{Dec}_{dk}(c =: (c_1, c_2))$: Compute $c_2 - dk^t c_1$ and output 0 iff its absolute value is less than $q/4$, i.e., output 0 iff

$$|c_2 - dk^t c_1 \bmod q| < q/4.$$

   - For proving security it is useful to introduce variant of Decision LWE called "short-secret" Decision LWE, where the secret $\bar{x}$ is also chosen from the error distribution. Thus LWE assumption is very versatile. In Assignment 3, you will show that the two assumptions are equivalent.
     - **Definition 2 (Short-Secret Decision LWE Problem [1]).**
     - *Input:*
       - Distribution 0: output $(\bar{A}, \bar{v})$, where $\bar{A} \leftarrow \mathbb{Z}_q^{n \times m}$ and $\bar{v} := \bar{x}^t \bar{A} + \bar{e}$ for $\bar{x}, \bar{e} \leftarrow E$

        - Distribution 1: output $(\bar{A}, \bar{v}) \leftarrow \mathbb{Z}_q^{n \times m} \times \mathbb{Z}_q^m$
    - *Decision:* Distribution 0 or 1?
  - **Exercise 3.** Show that Construction 3 is a secure PKE under Short-Secret Decision LWE assumption. (*Hint:* you have to rely on short-secret Decision LWE assumption many times)
- Additively-Homomorphic PKE
  - Finally, we will see that Regev PKE is additively homomorphic, i.e., given ciphertexts corresponding to two messages, it is possible to generate a ciphertext corresponding to their sum *without having to decrypt*. Let us first define additively-homomorphic PKE.
    - **Definition 3 (Additively-Homomorphic PKE).**
      - Let $\Pi := (\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$ be a PKE with a message-space $\mathcal{M}$ that supports addition and a ciphertext-space $\mathcal{C}$. Π is said to be homomorphic if there is an efficient algorithm Eval with the following syntax:
        - $c \leftarrow \mathsf{Eval}_{ek}(c_1, c_2)$ the (possibly) randomised *homomorphic-evaluation* algorithm Eval takes as input two ciphertexts $c_1, c_2 \in \mathcal{C}$ under an encryption key ek and outputs a ciphertext c.
      - It additionally has the following requirement:
        - *Correctness of evaluation.* For any $\lambda \in \mathbb{N}$ and $m_1, m_2 \in \mathcal{M}$,

$$\Pr_{\substack{(ek,dk) \leftarrow \mathsf{KGen}(1^\lambda) \\ c_1 \leftarrow \mathsf{Enc}_{ek}(m_1), c_2 \leftarrow \mathsf{Enc}_{ek}(m_2)}} [\mathsf{Dec}_{sk}(\mathsf{Eval}_{pk}(c_1, c_2)) = m_1 + m_2] = 1.$$

    - **Claim 1.** Regev PKE is additively homomorphic.
    - *Proof.* Consider simply *component-wise* adding two Regev ciphertexts $c_1 := (c_{1,1}, c_{1,2})$ and $c_2 := (c_{2,1}, c_{2,2})$, which are encryptions of bits b_1 and b_2 under $ek = (\bar{A}, \bar{v} := \bar{x}^t \bar{A} + \bar{e})$, respectively. That is, Eval outputs $c := (c_1, c_2)$, where

$$c_1 := c_{1,1} + c_{2,1} \bmod q = \bar{A}(\bar{r}_1 + \bar{r}_2) \bmod q$$

      and

$$c_2 := c_{2,1} + c_{2,1} \bmod q = \bar{v}^t(\bar{r}_1 + \bar{r}_2) + (b_1 + b_2)\lfloor q/2 \rceil \bmod q.$$

      What happens when one decrypts c under dk? We first compute

$$\begin{aligned} c_2 - dk^t c_1 &= (\bar{v}^t(\bar{r}_1 + \bar{r}_2) + (b_1 + b_2)\lfloor q/2 \rceil) - \bar{x}\bar{A}(\bar{r}_1 + \bar{r}_2) \\ &= \bar{e}_1(\bar{r}_1 + \bar{r}_2) + (b_1 + b_2)\lfloor q/2 \rceil \bmod q \end{aligned},$$

      and check whether this value lies in the range $(-q/4, q/4)$. Note that this holds as long as $\bar{e}_1(\bar{r}_1 + \bar{r}_2) \in (-q/4, q/4)$. Since $\bar{r}_1, \bar{r}_2 \leftarrow \{0,1\}^m$, this holds if each of $\bar{e}'s$ component $e_i \in (-q/8(m+1), q/8(m+1))$, which is true if (say) $B := q/8(m+1)$.
    - **Pop Quiz 2.**
      1. What should the bound B be for the encryption scheme to support k additions?

    2. Is a fresh ciphertext distributed the same as an evaluated ciphertext?
    3. Are they distinguishable?
- **Exercise 4.**
    1. Show that Dual Regev and BPR PKEs are also additively homomorphic. In particular, specify the evaluation algorithm
    2. What should the bound B be in each case?

Further Reading

- Dual Regev PKE was proposed in [this](https://eprint.iacr.org/ 2007/432) paper
- LPR PKE was proposed in this paper

2026s-CS758-Lecture16

Topics

- Lattice trapdoors

Sources

1. CS 294-168 Lattices, Learning with Errors and Post-Quantum Cryptography, Lecture 6, by Vaikuntanathan

Plan

- Recall/Plan Today
  - In the last two lectures:
    - We introduced the LWE problem and constructed an SKE under Decision LWE assumption. The key trick we learned was how to deal with noise.
    - We then constructed our first advanced cryptographic primitive from lattices: we lifted the construction of SKE to obtain a PKE from Decision LWE. The key idea we exploited for construction was that it is possible for Bob (the sender) to generate "fresh" random LWE samples *without* the knowledge of the secret key.
    - The main tool we used for analysis is the Leftover Hash Lemma (LHL). Let us recall it since it will play a key role today as well. Informally it states that hashing (using a random key) a random variable of sufficient min-entropy results in an output distribution that is statistically close to uniform. More formally:
      - **Lemma 1 (Lemma 1 in Lecture 15, rephrased).** Let P be a probability distribution over $\mathbb{Z}^m$, and let $H_\infty(P)$ denote its min-entropy, i.e.,

        $$H_\infty(P) := -\log \max_{\bar{v} \in \mathbb{Z}^m} P(\bar{v}).$$

        Moreover, let $\bar{A} \leftarrow \mathbb{Z}_q^{n \times m}$, with $m \geq n\ell + 2\lambda$, where $\ell := \lceil \log(q) \rceil$ (throughout this lecture) and $\bar{u} \leftarrow \mathbb{Z}_q^n$. If

        $$H_\infty(P) \geq m$$

        then for $\bar{r} \leftarrow P$ we have

        $$\mathsf{SD}((\bar{A}, \bar{A}\bar{r} \bmod q), (\bar{A}, \bar{u})) \leq 2^{-\lambda}.$$

  - In the coming two lectures we will construct two more advanced primitives from lattices: digital signatures and identity-based encryption.

- Key to constructing these will be the notion of *lattice trapdoors*, which we focus on today. We will first recall the notion of trapdoor (one-way) function, and then construct lattice trapdoors, in particular for lattice underlying SIS and LWE.

- Trapdoor (One-Way) Function (TDF)
  - Informally a trapdoor OWF is a OWF that is invertible *given* a secret trapdoor information (see **Figure 1**). More formally:
    - **Definition 1 (TDF).** An efficiently-computable function family

      $$f := \{f_k : \mathcal{D}_k \to \mathcal{R}_k\}_{k \in \mathcal{K} \subseteq \{0,1\}^*}$$

      with key-trapdoor sampling algorithm Gen is *trapdoor one-way* if it satisfies the following two requirements:

      1. *One-wayness.* For all PPT inverters Inv, the following is negligible

         $$\epsilon(\lambda) := \Pr_{\substack{(k,td) \leftarrow \mathsf{Gen}(1^\lambda) \\ x \leftarrow \mathcal{D}_k}}[\mathsf{Inv}(k, f_k(x)) \in f_k^{-1}(f_k(x))].$$

      2. *Invertibility given trapdoor.* There exists an efficient, deterministic inversion algorithm TDInv that inverts *given the trapdoor*, i.e.:

         $$\Pr_{\substack{(k,td) \leftarrow \mathsf{Gen}(1^\lambda) \\ x \leftarrow \mathcal{D}_k}}[\mathsf{TDInv}(k, td, f_k(x)) \in f_k^{-1}(f_k(x))] = 1$$

  - Let us look at a concrete example of TDF (in fact a trapdoor *permutation*) based on factorisation:
    - **Example 1.** For $N \in \mathbb{N}$, consider the function

      $$f_N : \mathbb{Z}_N^\times \to \mathbb{Z}_N^\times,$$

      defined as

      $$f_N(x) := x^3 \bmod N.$$

      For $N = PQ$ (sampled as in RSA scheme),

      $$3^{-1} \mod (P-1)(Q-1)$$

      acts as the trapdoor since

      $$(x^3)^{3^{-1}} = x \bmod N.$$

      Computing this trapdoor without the knowledge of the factors is believed to be hard.
  - However, TDFs are rare and not all OWFs are known to have trapdoors: e.g., we don't know a trapdoor for the DLP based OWF from Example 4 in Lecture 09.

- Lattice Trapdoors
  - **Pop Quiz 1.**
    - *Q.* What could constitute a trapdoor for lattices? We have already *implicitly* seen an example.

- *A.* Recall that the knowledge of a short vector $\bar{s}$ in $\bar{A}$'s kernel (e.g., via the SIS solver in the reduction from LWE to SIS) allowed us to solve Decision LWE. Thus short basis for the dual lattice is a good shout. More generally, short independent vectors in the dual lattice suffice.

- **Definition 2 (Definition 22 in [1]).** A matrix $\bar{S} = (\bar{s}_1, \cdots, \bar{s}_m) \in \mathbb{Z}^{m \times m}$ is a β-good lattice trapdoor for a matrix $\bar{A} \in \mathbb{Z}_q^{n \times m}$ if
  1. Each column vector in $\bar{S}$ is in the kernel of $\bar{A}$: i.e., $\bar{A}\bar{S} = 0^{n \times m} \bmod q$.
  2. Each column vector $\bar{s}_i$ in $\bar{S}$ is *short*, i.e., $\|\bar{s}_i\|_\infty \leq \beta$.
  3. $\bar{S}$ has *full rank* (over $\mathbb{R}^m$)
- It can be shown that $\det(\bar{A}) \leq q^n$ and by Minkowski's first theorem, there exist vectors in the dual lattice of length (in ℓ_∞ norm) at most $q^{n/m} \approx O(1)$. Thus we will aim for a constant β.
- Let us first see why a lattice trapdoor as in Definition 2 is useful by using it to solve Search LWE.
  - **Claim 1.** Given a lattice trapdoor $\bar{S}$ for $\bar{A}$, it is possible to invert the (injective) function

    $$f_{\bar{A}}(\bar{x}, \bar{e}) := \bar{x}^t \bar{A} + \bar{e}^t \bmod q.$$

  - *Proof.* Note that

    $$(\bar{x}^t \bar{A} + \bar{e}^t)\bar{S} = \bar{e}^t \bar{S} \bmod q.$$

    Since $\bar{e}$ and $\bar{S}$ have short entries, it can be shown that $\bar{e}\bar{S}$ has entries of absolute value at most $q/4$. As a result the modulo q operation has no effect. Thus it is possible to multiply with $\bar{S}^{-1}$ over reals (recall that $\bar{S}$ is full rank) to extract $\bar{e}$ and thus $\bar{x}$.
  - **Exercise 1.** Show how to use a lattice trapdoor to solve inhomogeneous SIS: i.e., invert Ajtai's function

    $$f_{\bar{A}}(\bar{e}) := \bar{A}\bar{e} \bmod q.$$

  - Thus the question we can now focus on is how to generate a random matrix $\bar{A}$ along with a short basis $\bar{S}$ for the dual lattice.
- *Sampling a Single Trapdoor Vector.* To this end, let us first focus on sampling a random matrix $\bar{A}$ with a *single* trapdoor vector $\bar{s}$. Let us figure our what goes wrong with the following obvious sampling procedure:
  - **Pop Quiz 2.**
    - *Q.* What about the following sampler: sample a random $\bar{A} \leftarrow \mathbb{Z}_q^{n \times m}$ and then sample a short $\bar{s} \in \mathbb{Z}_q^m$ such that $\bar{A}\bar{s} = 0^n \bmod q$? (Analogously think of sampling a random integer N and then outputting its factors.)
    - *A.* This is exactly the SIS problem. (In our analogy, it is exactly the integer factorisation problem.)

- Thus the moral of the story is that it is necessary to sample $\bar{A}$ and $\bar{s}$ *together* ! (And in our analogy N and its factors must be sampled together.) To this end, we are going to relax our requirement and sample $\bar{A}$ that is *statistically*-close to uniform.
- **Pop Quiz 3.**
  - *Q.* Using LHL, how do you sample $\bar{A} = (\bar{a}_1, \cdots, \bar{a}_m)$ along with a short $\bar{s}$ such that $\bar{A}\bar{s} = 0^n \bmod q$? Assume m is large enough.
  - *A.* Here is the procedure:
    - $\mathsf{Gen}(1^\lambda)$
      1. Sample $\bar{A}' \leftarrow \mathbb{Z}_q^{n\times(m-1)}$ and a uniformly random $\bar{s}' \leftarrow \{0,1\}^{m-1}$.
      2. Compute $\bar{a}_m := -\bar{A}'\bar{s}' \bmod q$ and output

         $$\bar{A} := \begin{pmatrix} \bar{A}' & \mid \bar{a}_m \end{pmatrix} \in \mathbb{Z}_q^{n\times m}$$

         and

         $$\bar{s} := \begin{pmatrix} \bar{s}' \\ 1 \end{pmatrix} \in \{0,1\}^m,$$

         i.e., *extend* $\bar{A}$' by appending the (minus of) *random subset sum* of the columns of $\bar{A}'$.
      - Let us see why the procedure is valid:
      1. First, observe that $\bar{s}$ is short since it consists of $0/1$ entries.
      2. Next, note that $\bar{A}\bar{s} = \bar{A}'\bar{s}' + \bar{a}_m = 0^n$ by construction
      3. Finally, it can be argued using LHL that $\bar{a}_m$ is statistically close to uniform as long as m is large enough (as per Lemma 1), and thus $\bar{A}$ is also statistically close to uniform.
- **Pop Quiz 4.**
  - *Q.* Is it possible to extend the above procedure and sample a *basis* $\bar{S}$? For example, why not pick $\bar{S}' \leftarrow \{0,1\}^{(m-1)\times(k+1)}$ and output

    $$\bar{A} := \begin{pmatrix} \bar{A}' & \mid -\bar{A}'\bar{S}' \bmod q \end{pmatrix} \in \mathbb{Z}_q^{n\times(m+k)}$$

    along with a short matrix

    $$\bar{S} := \begin{pmatrix} \bar{S}' \\ \bar{I}_{(k+1)} \end{pmatrix} \in \mathbb{Z}_q^{(m+k)\times(k+1)}.$$
  - *A.* No, since we rely on LHL, we need to start off with at least $n\ell + 2\lambda$ column vectors and we will never hit full rank -- there will always be that much deficit in the number of trapdoor vectors.

- Gadget Matrix and Inhomogeneous Trapdoor
  - The way around the bottleneck above is to use a slight twist of the approach, which suffices to generate trapdoors for LWE and SIS: instead of sampling $\bar{A}$ and

short *homogeneous* trapdoor $\bar{S} \in \mathbb{Z}^{m \times m}$ such that

$$\bar{A}\bar{S} = 0^{n \times m} \bmod q,$$

we sample $\bar{A}$ and a short *inhomogeneous* trapdoor $\bar{S} \in \mathbb{Z}^{m \times n\ell}$ such that

$$\bar{A}\bar{S} = \bar{G} \bmod q,$$

where

$$\bar{G} \in \mathbb{Z}^{n \times n\ell}$$

is the so-called "gadget matrix", which we define shortly. Why is an inhomogeneous trapdoor is useful?

1. First, it allows "reducing" the LWE problem with respect to $\bar{A}$ to the LWE problem with respect to $\bar{G}$ (but with slightly larger noise):

$$\bar{v}^t = \bar{x}^t\bar{A} + \bar{e}^t \bmod q \Rightarrow \bar{v}^t\bar{S} = \bar{x}^t\bar{A}\bar{S} + \bar{e}^t\bar{S} = \bar{x}^t\bar{G} + \bar{e}^t\bar{S} \bmod q,$$

   where $\bar{e}^t\bar{S}$ is short since $\bar{e}$ and $\bar{S}$ are.
2. Second, if we have a trapdoor $\bar{T}$ (in the sense of Definition 2) for $\bar{G}$, then we can solve LWE as shown in Claim 1. We will soon show that $\bar{G}$ does have such a trapdoor.

- To summarise, we use the inhomogeneous trapdoor $\bar{S}$ to first convert the LWE instance with respect to $\bar{A}$ into an LWE instance with respect to $\bar{S}$ and then use the homogeneous trapdoor $\bar{T}$ to extract the error and thus the secret. Let us finally define gadget matrix. Informally, it is matrix that encodes the "inverse" of bit decomposition function for $\mathbb{Z}_q$. More formally:
  - **Definition 3 (Gadget Matrix).** Let

$$\bar{g} := (1, 2, \cdots, 2^{\ell-1}) \in \mathbb{Z}^\ell$$

    denote the *gadget (row) vector*. The *gadget matrix* $\bar{G}$ is the block-diagonal matrix with gadget vector as its entries, i.e.:

$$\begin{pmatrix} \bar{g} & & & \\ & \bar{g} & & \\ & & \ddots & \\ & & & \bar{g} \end{pmatrix} \in \mathbb{Z}^{n \times n\ell}$$

  - Note that for an input $\bar{e} := (\bar{e}_1, \cdots, \bar{e}_n)^t \in (\{0,1\}^\ell)^n$, the output is $(\bar{g}\bar{e}_1, \cdots, \bar{g}\bar{e}_n)^t \in \mathbb{Z}_q^n$, where

$$\bar{g}\bar{e}_i = \sum_{j=0}^{\ell-1} e_{i,j} 2^j$$

    is simply the $\mathbb{Z}_q$ value that has $\bar{e}_i$ as the bit decomposition.
- *Homogeneous Trapdoor for $\bar{G}$.*

- **Pop Quiz 5.**
  - *Q.* Show that SIS is easy with respect to $\bar{G}$. As warm-up, come up with a short $\bar{s} \in [-2, 2]^\ell$ such that $\bar{g}\bar{s} = 0 \bmod q$.
  - *A.* Note that $\bar{s} := (2, -1, 0, \cdots, 0) \in [-2, 2]^\ell$ is short non-zero vector in $\bar{g}$'s kernel. It is now not hard to see that

$$\begin{pmatrix} \bar{s} \\ 0^{(n-1)\ell} \end{pmatrix} \in [-2, 2]^{n\ell}$$

    is a short non-zero vector in $\bar{G}$'s kernel.
- Given the above discussion, it is easy to see that the following matrix $\bar{T}'$ is a homogeneous trapdoor for $\bar{g}$ when $q = 2^\ell$ (i.e., is a power of 2):

$$\begin{pmatrix} 2 & & & & & \\ -1 & 2 & & & & \\ & -1 & \cdots & & & \\ & & \ddots & & & \\ & & & & 2 & \\ & & & & -1 & 2 \end{pmatrix} \in \mathbb{Z}^{\ell \times \ell}.$$

  In particular:
  1. $\bar{g}\bar{T}' = 0 \bmod q$
  2. Each column vector in $\bar{T}'$ is short
  3. $\bar{T}'$ has full rank since $\det(\bar{T}') = 2^\ell = q \neq 0$ (over integers)
- **Exercise 2.**
  1. What is the trapdoor for $\bar{g}$ for general q?
  2. What is the trapdoor for $\bar{G}$?
- *Sampling $\bar{A}$ with Inhomogeneous Trapdoor.*
  - Given the discussion above, the sampler we used earlier can be tweaked to sample $\bar{A}$ with an inhomogeneous trapdoor as described below. Note that we crucially rely on the fact that its dimension is $m \times n\ell$, and thus we don't run into the earlier issue of there always being a deficit. Below, we set $m = 2n\ell + 2\lambda$.
    - $\mathsf{Gen}(1^\lambda)$
      1. Sample $\bar{A}' \leftarrow \mathbb{Z}_q^{n \times (n\ell + 2\lambda)}$ and $\bar{S}' \leftarrow \{0,1\}^{(n\ell+2\lambda) \times n\ell}$.
      2. Output

$$\bar{A} := \begin{pmatrix} \bar{A}' & | -\bar{A}'\bar{S}' + \bar{G} \bmod q \end{pmatrix} \in \mathbb{Z}_q^{n \times m}$$

         along with a short matrix

$$\bar{S} := \begin{pmatrix} \bar{S}' \\ \bar{I}_{n\ell} \end{pmatrix} \in \{0,1\}^{m \times n\ell}.$$

- Note that

$$\bar{A}\bar{S} = \bar{A}'\bar{S}' + \bar{I}_{n\ell}(-\bar{A}'\bar{S}' + \bar{G}) = \bar{G}.$$

Further Reading

-

2026s-CS758-Lecture17

Topics

- Digital Signatures from Lattice Trapdoors
- Discrete Gaussian Sampling

Sources

1. CS 294-168 Lattices, Learning with Errors and Post-Quantum Cryptography, Lecture 6, by Vaikuntanathan
2. Trapdoors for Hard Lattices and New Cryptographic Constructions, by Gentry, Peikert and Vaikuntanathan

Plan

- Recall/Plan Today
  - In the last lecture:
    - We introduced trapdoor (one-way) function (TDF), which is a one-way function that is invertible given some secret trapdoor information. We then defined the notion of *homogeneous* trapdoor for q-ary lattices defined by a matrix $\bar{A} \in \mathbb{Z}_q^{n \times m}$ as a short basis $\bar{S} \in \mathbb{Z}^{m \times m}$ for the dual lattice $\Lambda_q^{\perp}(\bar{A})$. We then saw how to use $\bar{S}$ to solve LWE and SIS (which was left as an exercise).
    - We then tried to construct such trapdoors. We saw that Ajtai's procedure to sample an almost SIS matrix $\bar{A}$ along with short vectors by exploiting LHL doesn't quite yield a homogeneous trapdoor. But to get around this issue, we introduced the notion of *inhomogeneous* trapdoor with respect to a gadget matrix $\bar{G} \in \mathbb{Z}^{n \times n\ell}$, where $\ell := n\lceil \log q \rceil$, as a short matrix $\bar{S} \in \mathbb{Z}^{m \times n\ell}$ such that

      $$\bar{A}\bar{S} = \bar{G} \bmod q.$$

      We then saw how such a trapdoor allows *reducing* LWE/SIS instances (left as an exercise) with respect to $\bar{A}$ into LWE/SIS instances with respect to $\bar{G}$ and that LWE/SIS is easy with respect to $\bar{G}$ since it allows a homogeneous trapdoor.
  - In today's lecture:
    - We will see our first application of lattice trapdoor, viz., to construct lattice-based digital signature (DS) scheme, which is post-quantum secure. To be specific, we will describe "GPV Signature". In fact Falcon, one of the winners of the recently-concluded NIST post-quantum standardisation program, was built on top of GPV Signature.
    - To this end, we will review DSs and the "hash-then-invert" paradigm for constructing DSs. We will see that to instantiate this paradigm using lattices,

we will need a variant of TDF called "preimage-sampleable" TDF.

- Digital Signatures
  - Let's first review the definition of DSs:
    - **Definition 1 (DS).**
      - *Syntax:* a triple of algorithms $(\mathsf{KGen}, \mathsf{Sign}, \mathsf{Verify})$
        - $(sk, vk) \leftarrow \mathsf{KGen}(1^\lambda)$: the randomised *key-generation algorithm* KGen takes as input a security parameter λ in unary, and outputs a private signing key sk and a public verification key vk.
        - $\sigma \leftarrow \mathsf{Sign}_{sk}(\mu)$: the (possibly-randomised) *signing algorithm* Sign takes as input a message $\mu \in \{0,1\}^*$ and outputs a signature σ under sk.
        - $b := \mathsf{Verify}_{vk}(\sigma, \mu)$: the deterministic *signature-verification algorithm* Verify takes as inputs a signature σ and a verification key vk, and outputs a bit b indicating "accept" or "reject".
      - *Requirements:*
        - *Correctness:* for every λ and $\mu \in \{0,1\}^*$,

$$\Pr_{\substack{(sk,vk)\leftarrow\mathsf{KGen}(1^\lambda)\\ \sigma\leftarrow\mathsf{Sign}_{sk}(\mu)}}[\mathsf{Verify}_{vk}(\sigma,\mu)=1]=1.$$

        - *q-Existential-unforgeability under chosen-message attack (EU-CMA):* No efficient adversary F (i.e., a forger), having seen signatures on at most q messages of its choice, can *forge* a signature on a *fresh* message, i.e., for every PPT (oracle-aided) F that makes at most q queries to its oracle,

$$\epsilon(\lambda) := \Pr_{(sk,vk)\leftarrow\mathsf{KGen}(1^\lambda)}[(\sigma^*,\mu^*)\leftarrow \mathsf{F}^{\mathsf{Sign}_{\mathsf{sk}}(\cdot)}(vk), \mathsf{Verify}_{vk}(\sigma^*,\mu^*)=1, \mu^*\neq\mathcal{Q}]$$

          is negligible, where $\mathcal{Q}$ denotes the *set of queries* made by A to the $\mathsf{Sign}_{sk}(\cdot)$ oracle (see **Figure 1**).
  - *How to Construct Digital Signatures?*
    - In theory, OWFs are known to be sufficient (and necessary) to construct digital signatures. Therefore, in theory, post-quantum digital signatures are implied by SIS.
    - But this approach is inefficient, and there are more efficient approaches, under stronger cryptographic assumptions. The two prominent approaches are
      1. From identification schemes via Fiat-Shamir Transform. E.g., Schnorr Signature and CRYSTALS-Dilithium are constructed using this approach.
      2. From trapdoor permutations (TDP) via random oracles: the so-called "hash-then-invert" paradigm. E.g. Full-Domain Hash (FDH) follows this paradigm.
- Digital Signatures from Lattice Trapdoors

- In this lecture, we adopt the second approach, which we review next.
- *Hash-then-Invert Paradigm.*
  - We work in the random-oracle model (ROM) where all parties have access to a random function (over appropriate domain and co-domain).
  - **Construction 1 (DS from TDP in ROM).** Let

    $$f := \{f_k : \mathcal{D}_k \to \mathcal{D}_k\}_{k \in \mathcal{K} \subseteq \{0,1\}^*}$$

    with associated algorithms $(\mathsf{Gen}, \mathsf{TDInv})$ be a TDP and let

    $$H := \{H_k : \{0,1\}^* \to \mathcal{D}_k\}_{k \in \mathcal{K} \subseteq \{0,1\}^*}$$

    be a random oracle. The Hash-then-Invert signature scheme is described below:
    - $(sk, vk) \leftarrow \mathsf{KGen}(1^\lambda)$: Run $(k, td) \leftarrow \mathsf{Gen}$ and output $sk := td$ and $vk := k$.
    - $\sigma \leftarrow \mathsf{Sign}_{sk}(\mu)$:
      1. Parse $td =: sk$.
      2. Output $\sigma := \mathsf{TDInv}_{td}(H_k(m))$ (see **Figure 2**)
    - $b := \mathsf{Verify}_{vk}(\sigma, \mu)$: Accept iff $f_k(\sigma) = H_k(\mu)$.
  - **Example 1 (RSA-FDH).** When we substitute f with RSA TDP

    $$f_N(x) := x^3 \bmod N,$$

    you get FDH.
- *DS from Lattice Trapdoors: First Attempt.*
  - Let's consider the scheme obtained by instantiating the Hash-then-Invert template with the lattice-based TDF from Lecture 16:
    - **Construction 2 (DS from Lattice Trapdoors: First Attempt).** Let

      $$f := \{f_{\bar{A}} : [-B, B]^m \to \mathbb{Z}_q^n\}_{\bar{A} \in \mathbb{Z}_q^{n \times m}}$$

      with associated algorithms $(\mathsf{Gen}, \mathsf{TDInv})$ be a lattice-based TDF (for now we don't specify which one exactly) and let

      $$H := \{H_{\bar{A}} : \{0,1\}^* \to [-B, B]^m\}_{\bar{A} \in \mathbb{Z}_q^{n \times m}}.$$

      The DS scheme is described below:
      - $(sk, vk) \leftarrow \mathsf{KGen}(1^\lambda)$: Run Gen to obtain $(\bar{A}, \bar{S})$ and output $sk := (\bar{A}, \bar{S})$ and $vk := \bar{A}$.
      - $\sigma \leftarrow \mathsf{Sign}_{sk}(\mu)$:
        1. Parse $(\bar{A}, \bar{S}) =: sk$.
        2. Compute hash $\bar{h} := H_{\bar{A}}(\mu)$ and inverse $\bar{e} := \mathsf{TDInv}_{\bar{S}}(\bar{h})$, i.e., $\bar{e}$ is short and satisfies

           $$\bar{A}\bar{e} = \bar{h} \bmod q.$$

        3. Output $\sigma := \bar{e}$.

- $b := \mathsf{Verify}_{vk}(\sigma, \mu)$:
  1. Parse $\bar{A} =: vk$ and $\bar{e} =: \sigma$
  2. Accept iff $\bar{A}\bar{e} = H_{\bar{A}}(\mu)$ *and* $\bar{e}$ is short.

- **Pop Quiz 1.**
  1. *Q.* Is Construction 2 EU-CMA secure when $q = 0$?
     - A. Yes, this is essentially the ISIS problem.
  2. *Q.* Is it EU-CMA secure for arbitrary qs?
     - A. No! There is an attack in case the forger obtains m signatures $\sigma_1, \cdots, \sigma_m$ on (arbitrary) messages $\mu_1, \cdots, \mu_m$. To see why, let us focus on the particular case of *inhomogeneous* trapdoors, i.e., where $\bar{A}$ and $\bar{S}$ are sampled such that

$$\bar{A} := \begin{pmatrix} \bar{A}' & | -\bar{A}'\bar{S}' + \bar{G} \bmod q \end{pmatrix} \in \mathbb{Z}_q^{n \times m},$$

$$\bar{S} = \begin{pmatrix} \bar{S}' \\ \bar{I}_{n\ell} \end{pmatrix}$$

and, as a result, $\bar{A}\bar{S} = \bar{G} \bmod q$. Recall (from Lecture 16, Exercise 1) how we can use this trapdoor to solve a SIS instance $(\bar{A}, \bar{h})$:

1. Compute short $\bar{e}'$ such that $\bar{G}\bar{e}' = \bar{h}$ (recall that SIS is easy with respect to $\bar{G}$)
2. Output $\bar{S}\bar{e}'$, which is a short preimage of $\bar{s}$ under $\bar{A}$ since

$$\bar{A}(\bar{S}\bar{e}') = \bar{A}'\bar{S}'\bar{e}' + \bar{I}_{n\ell}(-\bar{A}'\bar{S}' + \bar{G})\bar{e}' = \bar{G}\bar{e}' = \bar{h} \bmod q.$$

Note that for this trapdoor, the signature-message pairs obtained by the adversary each satisfy

$$\sigma_i = \bar{e}_i = \begin{pmatrix} \bar{S}' \\ \bar{I}_{n\ell} \end{pmatrix} \bar{e}_i',$$

where the values of $\bar{e}_1', \cdots, \bar{e}_m'$ satisfy

$$\bar{G}\bar{e}_i' = \bar{h}_i := H_{\bar{A}}(\mu_i)$$

as in Step 1. Thus, we have a system of m equations in m variables, which can be solved using, say, Gaussian Elimination to obtain $\bar{S}'$.

- *Preimage-sampleable TDFs and GPV Signatures.*
  - It seems the attack on Construction 2 arises due to TDInv being deterministic (note that hash-then-invert is proved secure assuming a *TDP* and there are no guarantees for TDFs). The way around is require the lattice trapdoor to be *preimage-sampleable*, which requires the existence of a *randomised* algorithm $\mathsf{PreSampler}$ such that the following two distributions are *statistically* indistinguishable (i.e., their statistical distance is negligible):

1. $(\bar{A}, \bar{e}, \bar{A}\bar{e} \bmod q)$, where $\bar{A} \leftarrow \mathbb{Z}_q^{n \times m}$ and $\bar{e} \leftarrow D_{\mathbb{Z}^m,s}$ is sampled from Discrete Gaussian of width s over $\mathbb{Z}^m$. Recall from Lecture 14 that this is the distribution obtained by restricting the m-dimension Gaussian to $\mathbb{Z}^m$, i.e.,

$$D_{\mathbb{Z}^m,s}(\bar{y}) := \rho_s(\bar{y})/\rho_s(\mathbb{Z}^m).$$

2. $(\bar{A}, \bar{e}, \bar{h})$, where $\bar{A} \leftarrow \mathbb{Z}_q^{n \times m}$, $\bar{h} \leftarrow \mathbb{Z}_q^n$ and $\bar{e} \leftarrow \mathsf{PreSampler}_{\bar{S}}(\bar{A}, \bar{h})$.

- GPV Signature proceeds essentially as in Construction 2 but using $\mathsf{PreSampler}$ in place of TDInv. For a formal proof, we actually require something slightly stronger that described above: you can refer to [1,2] for more details. What remains to describe is how to sample from Discrete Gaussian, which we now do briefly.

- *Sampling from Discrete Gaussian.*
  - We will focus on $m = 1$, and refer the readers to [1,2] for details on higher dimensions. A natural first attempt is to first sample from the continuous Gaussian and then round the sample to the nearest integer. That is:
    - **Algorithm 1 (Sampler for 1D Discrete Gaussian: First Attempt).**
      - *Parameter.* Width $s \in \mathbb{N}$
      - *Algorithm.*
        1. Sample (real) $v \leftarrow N_s$
        2. Output $\lfloor v \rceil$
  - However, the above sampler does not work since its statistical distance from the discrete Gaussian can be shown to be non-negligible (as long as s is polynomial):
    - **Exercise 1.** Show that the statistical distance between Discrete Gaussian and output distribution of Algorithm 1 is $\approx 1/s^3$.
  - What works, however, is the following approach based on rejection sampling.
    - **Algorithm 2 (Sample for 1D Discrete Gaussian).**
      - *Parameter.* Width $s \in \mathbb{N}$ and statistical parameter $\lambda \in \mathbb{N}$
      - *Algorithm.*
        1. Sample random *integer* $y \leftarrow [-s\log(\lambda), s\log(\lambda)]$
        2. Toss a coin b that is 1 with probability $\rho_s(y) := e^{-\pi y^2/s^2}$ and is 0 with probability $1 - \rho_s(y)$.
        3. If $b = 1$, output y, otherwise go to Step 1.
    - To show that Algorithm 2 works, we need to show that:
      1. The probability that $D_{\mathbb{Z},s}$ outputs an integer outside $[-s\log(\lambda), s\log(\lambda)]$ is negligible. Intuitively, this follows by properties of (continuous) Gaussian distribution we came across earlier.
      2. Algorithm 2 terminates in polynomial time. This can be argued using Markov's inequality.
    - **Exercise 2.** Prove 1 and 2.

Further Reading

- Two lattice-based signatures have been standardised by NIST: Falcon and Dilithium
  - Falcon is based on the GPV signatures, but with NTRU lattices
  - Dilithium is based on the FST paradigm

2026s-CS758-Lecture18

Topics

- Identity-based encryption from lattice trapdoors

Sources

1. CS 294-168 Lattices, Learning with Errors and Post-Quantum Cryptography, Lecture 7, by Vaikuntanathan

Plan

- Recall/Plan
  - In the last two lectures:
    - We introduced homogeneous and inhomogeneous lattice trapdoors
    - We described GPV signature, which is a digital signature scheme in the random-oracle model (ROM), constructed using *preimage-sampleable* (lattice) trapdoors
      - Recall that (loosely-speaking) a preimage-sampleable trapdoor supports a *randomised* $\mathsf{PreSampler}$ algorithm such that the following two distributions are *statistically* indistinguishable:
        1. $(\bar{A}, \bar{e}, \bar{A}\bar{e} \bmod q)$, where $\bar{A} \leftarrow \mathbb{Z}_q^{n \times m}$ and $\bar{e} \leftarrow D_{\mathbb{Z}^m, s}$ is sampled from Discrete Gaussian of width $s \approx \beta \cdot \omega(\sqrt{\lambda})$ over $\mathbb{Z}^m$.
        2. $(\bar{A}, \bar{e}, \bar{h})$, where $\bar{A} \leftarrow \mathbb{Z}_q^{n \times m}$, $\bar{h} \leftarrow \mathbb{Z}_q^n$ and $\bar{e} \leftarrow \mathsf{PreSampler}_{\bar{S}}(\bar{A}, \bar{h})$.
      - The signature scheme then proceeds as follows:
        - **Construction 1 (DS from Preimage-Sampleable TDF).**
          - *Components:*
            1. A *preimage-sampleable TDF*

$$f := \{f_{\bar{A}} : [-B, B]^m \to \mathbb{Z}_q^n\}_{\bar{A} \in \mathbb{Z}_q^{n \times m}}$$

               with associated algorithms $(\mathsf{Gen}, \mathsf{PreSampler})$.
            2. A *hash function*

$$H := \{H_{\bar{A}} : \{0,1\}^* \to [-B, B]^m\}_{\bar{A} \in \mathbb{Z}_q^{n \times m}}$$

               (modelled as a RO).
          - *Construction:*
            - $(sk, vk) \leftarrow \mathsf{KGen}(1^\lambda)$: Run Gen to obtain $(\bar{A}, \bar{S})$ and output $sk := (\bar{A}, \bar{S})$ and $vk := \bar{A}$.
            - $\sigma \leftarrow \mathsf{Sign}_{sk}(\mu)$:
              1. Parse $(\bar{A}, \bar{S}) =: sk$.

2. Compute hash $\bar{h} := H_{\bar{A}}(\mu)$ and a random preimage $\bar{e} \leftarrow \mathsf{PreSampler}_{\bar{S}}(\bar{A}, \bar{h})$, i.e., $\bar{e}$ is short and satisfies

$$\bar{A}\bar{e} = \bar{h} \bmod q.$$

3. Output $\sigma := \bar{e}$.

- $b := \mathsf{Verify}_{vk}(\sigma, \mu)$:
  1. Parse $\bar{A} =: vk$ and $\bar{e} =: \sigma$
  2. Accept iff $\bar{A}\bar{e} = H_{\bar{A}}(\mu)$ *and* $\bar{e}$ is short.

- In today's lecture:
  - We introduce an advanced cryptographic primitive called identity-based encryption (IBE)
  - We construct IBE from preimage-sampleable TDF. The main ingredients we use are:
    1. GPV signature from last lecture
    2. Dual Regev PKE from Lecture 15

- Identity-Based Encryption (IBE)
  - Traditional PKE requires maintaining an authenticated *directory*, which maps e-mail addresses to public keys (e.g., see keys.openpgp.org). Users can then look up the public key corresponding to an e-mail ID. This architecture has a number of issues, e.g.:
    1. Who maintains the directory and why should we trust this maintainer?
    2. How do we make sure that an adversary (e.g., NSA) cannot add a malicious entry to the directory?
  - IBE solves these issues by having the e-mail ID *itself* act the public key! If you think about it a bit, you'll realise that IBE requires a set up.
    - **Exercise 1.** Convince yourself that an IBE requires setting up public parameters.
  - Thus the setting for IBE is as follows (see **Figure 1**):
    - A trusted key-issuing authority Ted (e.g., in IITB, it could be CC) generates and publishes (e.g., over intranet) the "master" public key mpk, while keeping the underlying "master" secret key msk private.
    - A user Alice (e.g., Bhava) approaches Ted *out of band* (e.g., during joining formalities) to obtain the decryption key dk_{id_A} corresponding to her identity $id_A \in \{0,1\}^*$ (e.g., `bhavadharini@cse.iitb.ac.in`). Ted generates dk_{id_A} using msk and gives it to Alice.
    - Another user Bob (e.g., Priyanshu), can now send an encrypted message to Alice using just the knowledge of id_A and mpk.
    - Alice can decrypt the encrypted message using dk_{id}.
  - **Pop Quiz 1.**
    - *Q.* Can you think of any issues with the IBE setting?

- *A.* One major issue is the the key escrow problem: Since Ted can generate decryption key for all uses, he can decrypt *all* encrypted messages! This is a big brother like situation. But there are ways around this:
    - *Distribute* the master key generation process among many trusted key-issuing authorities so that we don't have to rely on a single authority.
    - Use a variant of IBE called registration-based encryption (RBE)
- *Other Applications of IBE:*
    - *Delegation of Decryption.* Suppose Alice is going on a holiday (e.g., on 20/Mar). Alice can ask senders to append date to her e-mail id (e.g., `alice@email.com|20/Mar`) while encrypting and then give her PA Polly the decryption key *only* for that date (i.e., for $id'_A :=$ `alice@email.com|20/Mar`) . Once Alice is back from vacation the next day, Polly can no longer decrypt. (You will hear more about this topic in Nilabha's and Bhava's presentation.)
    - *Black-Box Construction of CCA-PKE.* There is a generic construction of CCA-secure PKE from IBE.
- Before formally defining IBE, let us first think about the security model.
    - **Pop Quiz 2.**
        - *Q.* How would you model security? What should the adversary able to see?
        - *A.* The master secret key msk must be kept private for any hope. But the adversary could be given decryption keys for identities of its choice, as long it is different from the identity it attacks.
    - **Definition 1 (IBE, [1], Section 7.1).**
        - *Syntax:* a four-tuple of algorithms $(\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$
            - $(mpk, msk) \leftarrow \mathsf{Setup}(1^\lambda)$: the randomised *set-up algorithm* takes as input a security parameter λ in unary, and outputs a master public key mpk and a master secret key msk.
            - $dk_{id} \leftarrow \mathsf{KGen}(msk, id)$: the (potentially-randomised) *key-generation algorithm* KGen takes as inputs a master secret key msk and a user identity $id \in \{0,1\}^*$, and outputs a decryption key sk_{id} corresponding to id.
            - $c \leftarrow \mathsf{Enc}_{mpk}(id, b)$: the randomised *encryption algorithm takes as inputs a master public key mpk, a user identity id and a bit $b \in \{0,1\}$ and outputs a ciphertext c.
            - $b := \mathsf{Dec}_{sk_{id}}(c)$: the deterministic *decryption algorithm* takes as input a ciphertext c and a decryption key dk, and outputs a bit b.
        - *Requirements:*
            - *Correctness of decryption:* for every λ, $id \in \{0,1\}^*$ and $b \in \{0,1\}$,

$$\Pr_{\substack{(mpk,msk)\leftarrow\mathsf{Setup}(1^\lambda)\\ dk_{id}\leftarrow\mathsf{KGen}(msk,id)}}[\mathsf{Dec}_{dk_{id}}(\mathsf{Enc}_{mpk}(id,b)) = b] = 1.$$

- *Selective security:* No efficient adversary A, which commits *upfront* to the challenge identity id^*, can distinguish ciphertexts of 0 and 1 under id^*, *even given* the decryption key for *other* identities of its choice. That is, for every (stateful) PPT A, the probability of A winning the following game (see **Figure 2**) *away from half* is negligible:
  1. A commits to the challenge identity $id^* \in \{0,1\}^*$
  2. Run $(mpk, msk) \leftarrow \mathsf{Setup}(1^\lambda)$ and send mpk to A.
  3. A makes a series of key-generation queries: as long as the queried identity id is not id^*, respond with $dk_{id} \leftarrow \mathsf{KGen}(msk, id)$.
  4. Toss a bit b^* and send the challenge ciphertext $c^* \leftarrow \mathsf{Enc}_{mpk}(id^*, b^*)$ to A.
  5. A *wins* if it correctly guesses b^*

- *How to Construct IBE?*
  - IBEs were first constructed based on hardness of cryptographic pairings and quadratic-residuosity problem. However, these IBEs are not post-quantum secure.
  - The first post-quantum secure IBE (in ROM) is the "GPV IBE", which is what we construct today. Looking ahead it relies on preimage-sampleable TDF and is hence based on lattices.
  - We now know how to construct IBE from DDH.

- IBE from Lattices: GPV IBE
  - To get an insight into the GPV-IBE construction, let us first try to solve the following problem:
    - **Pop Quiz 3.**
      - *Q.* Construct a digital signature from IBE.
      - *A.* The idea is to treat the message to be signed as an identity and use the IBE key-generation algorithm for signing: existential unforgeability of the signature scheme follows security of the IBE. In more detail:
        - **Construction 2 (Due to Moni Naor)**
          - The master secret key serves as the signing key and the master public key serves as the verification key
          - The signature σ on a message $\mu \in \{0,1\}^*$ is $\sigma \leftarrow \mathsf{KGen}(msk, \mu)$
          - To verify σ against μ, encrypt (say) $b = 0$ under (mpk, μ) and accept iff the resulting ciphertext decrypts to 0 under σ.
    - **Exercise 2.**
      1. Argue why the DS scheme is correct.
      2. Show that the DS scheme is EU-CMA secure if the IBE is secure. Does the IBE being selectively secure suffice? Or do you need stronger security from the IBE?

- Given this connection between IBE and DS, it is natural to ask whether the converse also holds: i.e., is it possible to construct an IBE from DS and (say) PKE? While this does not hold in general, we will see that for *specific* lattice-based constructions, this can be made to work. To be specific, the GPV IBE is constructed by putting together GPV signature and Dual Regev PKE following the above template: i.e.,
    - The IBE key generation involves signing the (hashed) identity using GPV signature.
    - The IBE encryption involves encrypting the message using Dual Regev PKE with the (hashed) identity as the public key.
- Before describing the GPV IBE construction in detail, we recall Dual Regev PKE from Lecture 15.
    - "*Dual Regev*" *PKE.*
        - **Construction 3 ("Dual Regev" PKE, Construction 2 in Lecture 15).**
            - $(ek, dk) \leftarrow \mathsf{KGen}(1^\lambda)$: Sample $\bar{A} \leftarrow \mathbb{Z}_q^{n \times m}$ with $m \geq 2n\lceil \log(q) \rceil$ and $\bar{r} \leftarrow \{0,1\}^m$. Output $ek := (\bar{A}, \bar{v} := \bar{A}\bar{r})$ and $dk := \bar{r}$.
            - $c \leftarrow \mathsf{Enc}_{ek}(b)$: to encrypt $b \in \{0,1\}$, sample $\bar{x} \leftarrow \mathbb{Z}_q^n$, $\bar{e}, e \leftarrow E$ and output $c := (c_1, c_2)$ where

              $$c_1 := \bar{x}^t \bar{A} + \bar{e}$$

              and

              $$c_2 := \bar{x}^t \bar{v} + e + \lfloor q/2 \rceil b \bmod q.$$

            - $b := \mathsf{Dec}_{dk}(c =: (c_1, c_2))$: Compute $c_2 - dk^t c_1$ and output 0 iff its absolute value is less than $q/4$, i.e., output 0 iff

              $$|c_2 - dk^t c_1 \bmod q| < q/4.$$

        - Recall the differences between Regev and Dual Regev PKEs:
            - In Regev PKE:
                1. The public key is *not* uniformly random due to $\bar{v}$ being sampled according to LWE distribution.
                2. The secret key $\bar{x}$ corresponding to a public key $(\bar{A}, \bar{v})$ is *unique* (since $\bar{e}$ is chosen so that it has norm at most minimum distance).
                3. Different random coins may yield the same ciphertext since there are collisions in the map $\bar{r} \mapsto \bar{A}\bar{r}$.
            - In "Dual Regev" PKE the *roles of secret key and random coins* are swapped and thus it ends up having the following properties:
                1. The public key *is uniform* (which is key to our application).

2. There are many valid secret keys corresponding to a public key.
3. A ciphertext corresponds to a *unique random coin*.

- *GPV IBE.* Let us now put together GPV IBE:
  - **Construction 3 ([1], Section 7.4).**
    - *Components:* same as Construction 1.
    - *Construction:*
      - $(mpk, msk) \leftarrow \mathsf{Setup}(1^\lambda)$: Run Gen to obtain $(\bar{A}, \bar{S})$ and output $msk := (\bar{A}, \bar{S})$ and $mpk := \bar{A}$.
      - $dk_{id} \leftarrow \mathsf{KGen}(msk, id)$:
        1. Parse $(\bar{A}, \bar{S}) =: msk$.
        2. Compute hash $\bar{h} := H_{\bar{A}}(\mu)$ and a random preimage $\bar{e} \leftarrow \mathsf{PreSampler}_{\bar{S}}(\bar{A}, \bar{h})$, i.e., $\bar{e}$ is short and satisfies
        $$\bar{A}\bar{e} = \bar{h} \bmod q. \tag{1}$$
        3. Output $dk_{id} := \bar{e}$.
      - $c \leftarrow \mathsf{Enc}_{mpk}(id, b)$: to encrypt $b \in \{0, 1\}$, sample $\bar{x} \leftarrow \mathbb{Z}_q^n$, $\bar{e}, e \leftarrow E$ and output $c := (c_1, c_2)$ where
        $$c_1 := \bar{x}^t \bar{A} + \bar{e}$$
        and
        $$c_2 := \bar{x}^t \bar{h} + e + \lfloor q/2 \rceil b \bmod q$$
        for $\bar{h}$ computed as in (1).
      - $b := \mathsf{Dec}_{dk_{id}}(c)$: Compute $c_2 - dk_{id}^t c_1$ and output 0 iff its absolute value is less than $q/4$, i.e., output 0 iff
        $$|c_2 - dk_{id}^t c_1 \bmod q| < q/4.$$
  - **Pop Quiz 4.**
    - *Q.* Can anyone point out why we need to switch to Dual Regev instead of just using Regev?
    - *A.* So that we can rely on normal RO.
  - It can be shown that Construction 3 is a selectively secure IBE in the ROM. We state the theorem without a proof (which you can find in [1]).
    - **Theorem 1.** If f is a preimage-sampleable TDF, then Construction 3 is a selectively-secure IBE in the ROM.

Further Reading

- Boneh-Franklin IBE: Identity-Based Encryption from the Weil Pairing
- Cocks IBE: An Identity Based Encryption Scheme Based on Quadratic Residues

- ABB IBE: Efficient Lattice (H)IBE in the Standard Model
- Döttling-Garg IBE: Identity-Based Encryption from the Diffie-Hellman Assumption
- CCA-PKE from IBE: Chosen-Ciphertext Security from Identity-Based Encryption

2026s-CS758-Lecture19

Topics

- Fully-homomorphic encryption (FHE)
- (Levelled) FHE from LWE: Gentry-Sahai-Waters FHE

Sources

1. Homomorphic Encryption, by Halevi

Plan

- Recall/Plan
  - In the last lecture:
    - We introduced identity-based encryption (IBE), the first (truly) advanced cryptographic primitive in our course. In particular, we constructed GPV IBE by "composing" GPV signature from Lecture 17 with Dual Regev PKE from Lecture 15.
  - In today's lecture:
    - We introduce fully-homomorphic encryption (FHE), the second advanced cryptographic primitive we will cover. FHE allows carrying out *any* computation on encrypted data, without having to decrypt it first. This leads to some very interesting applications:
      - *Secure delegation/outsourcing of computation.* Suppose Alice has a weak device (e.g., a laptop) and wants to use GCP to carry out a heavy computation F. For concreteness, suppose Alice is a medical researcher and wants to generate statistics on patient data D. But due to privacy laws (like GDPR or DPDPA), Alice cannot simply outsource this computation to GCP by sending over D in the clear. FHE allows Alice to encrypt D, use GCP to *homomorphically* compute the statistics $F(D)$, and then retrieve $F(D)$ by decryption (see **Figure 1**).
      - *Private-information retrieval (PIR).* A server holds a large database D and a user Alice wants to retrieve an entry $D[i]$ from this database *without* the server learning i. Alice, e.g., can send an encryption of i to the server, who can then homomorphically compute $D[i]$, and return the encrypted result to Alice.
      - *Secure multi-party computation (SMPC).* SMPC allows n parties, each holding an input D_i, to compute a n-argument function $F(D_1, \cdots, D_n)$ in a manner that each party i learning nothing about the input of the remaining parties, except the output. FHE offers a generic way to construct SMPC.

- Recall Regev PKE from Lecture 15.
  - **Construction 1 (Regev PKE, Construction 1 in Lecture 15, rephrased).**
    - $(ek, dk) \leftarrow \mathsf{KGen}(1^\lambda)$: Let $n, m, q \in \mathbb{N}$ and error distribution E over $\mathbb{Z}$ be LWE parameters, with $n := O(\lambda)$ and $m := n\lceil \log(q) \rceil + 2\lambda$. Output

      $$ek := \bar{A}' := \begin{pmatrix} \bar{A} \\ \bar{v}^t \end{pmatrix} \in \mathbb{Z}_q^{n \times m}$$

      and $dk := \bar{x}' := (-\bar{x}^t, 1) \in \mathbb{Z}_q^n$, where
      - $\bar{A} \leftarrow \mathbb{Z}_q^{(n-1)\times m}$
      - $\bar{v}^t := \bar{x}^t \bar{A} + \bar{e}^t$ for $\bar{x} \leftarrow \mathbb{Z}_q^{n-1}$ and $\bar{e} \leftarrow E^m$.
    - $c \leftarrow \mathsf{Enc}_{ek}(b)$: to encrypt $b \in \{0,1\}$, sample $\bar{r} \leftarrow \{0,1\}^m$ and output

      $$c := \bar{A}'\bar{r} + (0, \cdots, 0, \lfloor q/2 \rceil b)^t \bmod q \in \mathbb{Z}_q^n.$$

    - $b := \mathsf{Dec}_{dk}(c)$: Output 0 iff

      $$|dk \cdot c \bmod q| < q/4.$$

  - Recall that Regev PKE supports homomorphic addition: adding two (or more) ciphertexts coordinate-wise yields a ciphertext of sum of the two (or more) underlying plaintexts. Today, we build on Regev PKE to construct an FHE scheme that supports evaluation of any function representable by a *bounded-depth* circuit. Such an FHE is called "levelled" FHE. In the next lecture, we will remove this restriction using a technique called *bootstrapping* to get "full" FHE.
- Fully-Homomorphic Encryption (FHE)
  - Let us first formally define FHE: for this lecture, we restrict our attention to bit PKE and homomorphically computing n-input Boolean functions $F : \{0,1\}^n \to \{0,1\}$ -- extending to the general case is not too difficult.
    - **Definition 1 ([1]).** A bit PKE scheme $\Pi = (\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$ supports *homomorphic evaluation* over a circuit class $\mathcal{F}$ if it additionally supports an efficient algorithm Eval with the following syntax:
      - $c \leftarrow \mathsf{Eval}_{ek}(\bar{c}, F)$: the (possibly) randomised *homomorphic-evaluation* algorithm Eval takes as input a vector of ciphertexts $\bar{c} \in \mathcal{C}^n$ under an encryption key ek and the circuit description of a function $F : \{0,1\}^n \to \{0,1\} \in \mathcal{F}$ and outputs an "evaluated" ciphertext c.
    - It must additionally satisfy *correctness of homomorphic evaluation:* for any $\lambda \in \mathbb{N}$, input bits $b_1, \cdots, b_n \in \{0,1\}$ and $F \in \mathcal{F}$

      $$\Pr_{\substack{(ek,dk) \leftarrow \mathsf{KGen}(1^\lambda) \\ c_1 \leftarrow \mathsf{Enc}_{ek}(b_1), \cdots, c_n \leftarrow \mathsf{Enc}_{ek}(b_n)}} [\mathsf{Dec}_{dk}(\mathsf{Eval}_{ek}((c_1, \cdots, c_n), F) = F(b_1, \cdots, b_n))] = 1.$$

    - **Pop Quiz 1.**

        - *Q.* Is correctness of homomorphic evaluation (in addition correctness of decryption and ciphertext indistinguishability) sufficient for interesting FHE instantiations?
        - *A.* No! Consider, e.g., the pathological scheme in which Eval simply outputs $(\bar{c}, F)$, and we modify the decryption algorithm to individually decrypt the ciphertexts in $\bar{c}$ to obtain $b_1, \cdots, b_n$ and then output $F(b_1, \cdots, b_n)$. The scheme is a valid FHE according to our current definition.
    - **Pop Quiz 2.**
        - *Q.* How can we rule out such trivial schemes?
        - *A.* These can be ruled out by additionally requiring any one of the following requirements:
            - Circuit privacy: i.e., the evaluated ciphertext must not leak information about the circuit F that was evaluated; or
            - Compactness, which requires the *size* of an evaluated ciphertext *to not* grow with the size of F.
    - We will stick to compactness, formally defined below:
        - *Compactness:* The *size* of an evaluated ciphertext *must not* grow with the size of F. That is, there exists a fixed polynomial $B(\cdot)$ such that for any $\lambda \in \mathbb{N}$, input bits $b_1, \cdots, b_n \in \{0,1\}$ and $F \in \mathcal{F}$

$$\Pr_{\substack{(ek,dk) \leftarrow \mathsf{KGen}(1^\lambda) \\ c_1 \leftarrow \mathsf{Enc}_{ek}(b_1), \cdots, c_n \leftarrow \mathsf{Enc}_{ek}(b_n)}} [|\mathsf{Eval}_{ek}((c_1, \cdots, c_n), F)| \leq B(\lambda)] = 1.$$

- What are some of the interesting classes of circuits?
    - If $\mathcal{F}$ is the set of *all* polynomial-sized circuits, then the scheme is a (full) FHE.
    - If $\mathcal{F}$ is the set of all *bounded-depth* circuits, then the scheme is a *levelled* FHE.
- *How to Construct FHE?*
    - *Partially-homomorphic* PKEs, which support homomorphic addition or multiplication, have been known for a while:
        - Classical examples of *additively*-homomorphic PKE are Goldwasser-Micali and Pailler PKE, both of which are based on *number-theoretic* assumptions. In Lecture 15 we saw that Regev PKE too is additively homomorphic.
        - Examples of *multiplicatively*-homomorphic PKEs are (unpadded) RSA and Elgamal PKE.
    - However (unlike IBE) FHE is known only from lattices!
- FHE from LWE: GSW FHE
    - We will first focus on homomorphically adding and multiplying *two* ciphertexts -- this already introduces the core ideas in the GSW FHE scheme, and the extension to arbitrary computation then is not too difficult. We first review the basic linear

algebra notions the scheme relies on. Although the discussion below is for a general field $\mathbb{F}$, we will eventually work with $\mathbb{F} = \mathbb{Z}_q$.

- *Eigenvectors and Eigenvalues*.
  - **Definition 2.** Let $\mathbb{F}$ be a field and let $\bar{C} \in \mathbb{F}^{n \times n}$ be a square matrix over $\mathbb{F}$. A vector $\bar{x} \in \mathbb{F}^n$ is a *left-eigenvector* of $\bar{C}$ with *eigenvalue* $v \in \mathbb{F}$ if

$$\bar{x}\bar{C} = v\bar{x}.$$

  - We will make extensive use of the following simple fact about eigenvalues and eigenvectors.
    - **Fact 1.** If $\bar{C}_1, \bar{C}_2 \in \mathbb{F}^{n \times n}$ have a common eigenvector $\bar{x}$ with eigenvalues v_1 and v_2, then:
      1. $\bar{x}$ is also an eigenvector of $\bar{C}_1 \pm \bar{C}_2$ with eigenvalue $v_1 \pm v_2$.
      2. $\bar{x}$ is also an eigenvector of $\bar{C}_1\bar{C}_2$ with eigenvalue $v_1 v_2$.
  - We will also need to (inverse) sample matrices *given* eigenvectors and eigenvalues:
    - **Exercise 1.** Given n linearly-independent vectors $\bar{x}_1, \cdots, \bar{x}_n \in \mathbb{F}^n$ and $u_1, \cdots, u_n \in \mathbb{F}$, construct a (random) matrix $\bar{C} \in \mathbb{F}^{n \times n}$ that has $\bar{x}_1, \cdots, \bar{x}_n$ as eigenvectors with eigenvalues $u_1, \cdots, u_n$ (respectively).
- *First Attempt: Eigenvector Encryption.*
  - As a first attempt, we are going to consider an (implicit) scheme where secret keys correspond to vectors in $\mathbb{Z}_q^n$ and ciphertexts correspond to matrices over $\mathbb{Z}_q^{n \times n}$. In particular, we are going to maintain the following invariant:
    - **Invariant 1.** A ciphertext $\bar{C}$ encrypts a message $v \in \mathbb{Z}_q$ under a secret $\bar{x}$ if $\bar{x}$ is an eigenvector of $\bar{C}$ with eigenvalue v, i.e.,

$$\bar{x}\bar{C} = v\bar{x} \bmod q. \tag{1}$$

  - By Fact 1, *functionality* holds:
    1. To homomorphically *add/subtract* two ciphertexts $\bar{C}_1$ and $\bar{C}_2$ encrypting v_1 and v_2 (respectively) under $\bar{x}$, simply add/subtract them!
    2. To homomorphically *multiply* two ciphertexts $\bar{C}_1$ and $\bar{C}_2$ encrypting v_1 and v_2 (respectively) under $\bar{x}$, simply multiply them!
  - **Pop Quiz 3.**
    - *Q.* What is wrong with the first attempt then?
    - *A.* It is *insecure*: it is *easy* to compute (e.g., using Gaussian elimination) the eigenvectors of a matrix $\bar{C}$ and thus extract the secret and thus the message.
- *Second Attempt: Noisy Eigenvector Encryption.*
  - A natural suggestion to fix the first attempt is to add noise, i.e., we are going to relax the invariant in (1) an "noisy" one $\bar{x}\bar{C} \approx v\bar{x} \bmod q$:
    - **Invariant 2.** A ciphertext $\bar{C}$ encrypts a message $v \in \mathbb{Z}_q$ under a secret $\bar{x}$ if $\bar{x}$ is an *approximate* eigenvector of $\bar{C}$ with eigenvalue v, i.e.,

$$\bar{x}\bar{C} = v\bar{x} + \bar{e} \bmod q,$$

for a "short" noise vector $\bar{e} \in \mathbb{Z}^n$.

- Since Gaussian elimination is "brittle" over finite fields, computing eigenvectors now seems hard, and thus the scheme *seems secure*. Let us check what has happened to functionality:
  1. To homomorphically add/subtract two ciphertexts $\bar{C}_1$ and $\bar{C}_2$ encrypting v_1 and v_2 (respectively) under $\bar{x}$, we can *still* simply add/subtract them since $\bar{x}$ is also an approximate eigenvector of $\bar{C}_1 \pm \bar{C}_2$ with eigenvalue $v_1 \pm v_2$: if $\bar{x}\bar{C}_1 = v_1\bar{x} + \bar{e}_1 \bmod q$ and $\bar{x}\bar{C}_2 = v_2\bar{x} + \bar{e}_2 \bmod q$ then

     $$\bar{x}(\bar{C}_1 \pm \bar{C}_2) = \bar{x}(v_1 \pm v_2) + (\bar{e}_1 \pm \bar{e}_2) \bmod q.$$

     Note that the error is $(\bar{e}_1 \pm \bar{e}_2)$ and has thus grown by a factor of two.
  2. However, when we multiply two ciphertexts $\bar{C}_1$ and $\bar{C}_2$ as above, they satisfy

     $$\begin{aligned}\bar{x}\bar{C}_1\bar{C}_2 &= (v_1\bar{x} + \bar{e}_1)\bar{C}_2 \\ &= v_1\bar{x}\bar{C}_2 + \bar{e}_1\bar{C}_2 \\ &= v_1(v_2\bar{x} + \bar{e}_2) + \bar{e}_1\bar{C}_2 \\ &= v_1v_2\bar{x} + (v_1\bar{e}_2 + \bar{e}_1\bar{C}_2) \bmod q.\end{aligned}$$

     Note that the error is $(v_1\bar{e}_2 + \bar{e}_1\bar{C}_2)$, but there is no longer a guarantee that it is short: while v_1 can be kept small by encrypting only small plaintext values (e.g., by only using Boolean gates), there is no way to control $\bar{C}_2$.
- *Final Attempt: Gadget Matrix to the Rescue.*
  - The way around the above issue is to keep the norm in check by multiplying two ciphertexts using $\bar{C}_1 G^{-1}(\bar{C}_2)$, where

    $$G^{-1} : \mathbb{Z}_q^{n\times n} \to \mathbb{Z}_q^{n\times n\ell}$$

    is the "bit-decomposition" operation, which we explain next. For

    $$\bar{g} := (1, 2, \cdots, 2^{\ell-1}) \in \mathbb{Z}^\ell,$$

    where $\ell := \lceil \log(q) \rceil$, recall that the *gadget matrix* $\bar{G} \in \mathbb{Z}^{n\times n\ell}$ is the block-diagonal matrix

    $$\begin{pmatrix} \bar{g} & & & \\ & \bar{g} & & \\ & & \ddots & \\ & & & \bar{g} \end{pmatrix} \in \mathbb{Z}^{n\times n\ell}.$$

    Recall also that for $\bar{r} \in \{0,1\}^{n\ell}$, the linear operation $\bar{G}\bar{r} \bmod q$ computes the $\mathbb{Z}_q^n$-element having the bit representation $\bar{r}$. The operation G^{-1} is its "inverse", i.e., it takes a vector or matrix over $\mathbb{Z}_q$ and outputs the corresponding bit

decomposition. Note that for any $\bar{x} \in \mathbb{Z}_q^n$,

$$\bar{G} \cdot G^{-1}(\bar{x}) = \bar{x}.$$

- To work with the gadget matrix, we need $\bar{C}_1, \bar{C}_2 \in \mathbb{Z}_q^{n \times n\ell}$ and the new invariant is:
  - **Invariant 3.** A ciphertext $\bar{C}$ encrypts a message $v \in \mathbb{Z}_q$ under a secret $\bar{x}$ if

    $$\bar{x}\bar{C} = v\bar{x}\bar{G} + \bar{e} \bmod q,$$

    for a "short" noise vector $\bar{e}$.
- Now we have

$$\begin{aligned}
\bar{x}\bar{C}_1 G^{-1}\bar{C}_2 &= (v_1\bar{x}\bar{G} + \bar{e}_1)G^{-1}\bar{C}_2 \\
&= v_1\bar{x}\bar{G}G^{-1}\bar{C}_2 + \bar{e}_1 G^{-1}\bar{C}_2 \\
&= v_1\bar{x}\bar{C}_2 + \bar{e}_1 G^{-1}\bar{C}_2 \\
&= v_1(v_2\bar{x}\bar{G} + \bar{e}_2)G^{-1}\bar{C}_2 + \bar{e}_1 G^{-1}\bar{C}_2 \\
&= v_1 v_2 \bar{x}\bar{G} + (v_1\bar{e}_2 + \bar{e}_1 G^{-1}\bar{C}_2) \bmod q.
\end{aligned}$$

- *Putting it All Together: GSW FHE.*
  - For ease of analysis, we will work with a circuit made up solely of NAND gates.
    - **Exercise 2.**
      1. Show that homomorphic NAND of two ciphertexts $\bar{C}_1$ and $\bar{C}_2$ as above can be computed as

         $$\bar{G} - \bar{C}_1 \times G^{-1}\bar{C}_2 \bmod q. \tag{2}$$

      2. When v_is are restricted to bits, show that the ℓ_∞-norm of the error grows by a factor of at most $n\ell + 1$.
  - It follows by Exercise 2.2 that if the initial noise bound for the error distribution is B, then repeating the homomorphic NAND operation d times leads to a noise bounded by $B \cdot (n\ell + 1)^d$. Keeping this in mind, the GSW (levelled) FHE for depth-d circuits is described below. In particular, we will maintain the following invariant:
    - **Invariant 4.** A ciphertext $\bar{C}$ at level i of the circuit encrypts a bit $b \in \{0, 1\}$ under the secret $\bar{x}$ if

      $$\bar{x}\bar{C} = b\bar{x}\bar{G} + \bar{e} \bmod q,$$

      for

      $$\|\bar{e}\|_\infty \leq nm \cdot (n\ell + 1)^i.$$

    - **Construction 2 (GSW FHE, Section 3.2.4 in [1])**.
      - $(ek, dk) \leftarrow \mathsf{KGen}(1^\lambda)$:

        - Set LWE parameters $n, m, q \in \mathbb{N}$ such that $m \geq n\ell$ and $nm(n\ell+1)^d < q/4$.
        - Output $ek := \bar{A}'$ and $dk := \bar{x}' := (-\bar{x}^t, 1)$, sampled as in Regev PKE
    - $c \leftarrow \mathsf{Enc}_{ek}(b)$: to encrypt $b \in \{0,1\}$, sample $\bar{R} \leftarrow \{0,1\}^{m \times n\ell}$ and output the ciphertext matrix

      $$\bar{C} := b\bar{G} + \bar{A}'\bar{R} \bmod q \in \mathbb{Z}_q^{n \times n\ell}.$$

      Note that since

      $$\begin{aligned} \bar{x}'\bar{C} &= b\bar{x}'\bar{G} + \bar{x}'\bar{A}'\bar{R} \\ &= b\bar{x}'\bar{G} + \bar{e}\bar{R}, \end{aligned}$$

      Invariant 4 for the case of $i = 0$ holds.
    - **Exercise 3.** How do you decrypt?
    - $c \leftarrow \mathsf{Eval}_{ek}(\bar{c}, F)$:
      1. For each NAND gate in F (topologically):
         1. Let $\bar{C}_1, \bar{C}_2$ denote the ciphertexts corresponding to its input gates
         2. Compute $\bar{C}$ as in (2).
      2. Output $\bar{C}$ corresponding to the output gate (see **Figure 2**).
- **Theorem 1 (Theorem 4 in [1], simplified).** Construction 2 is a (weakly) compact FHE for bounded-depth (NAND) circuits under Decision LWE assumption.
    - You can read more about how to precisely set the parameters and what exactly you need from the LWE assumption in [1].

Further Reading

- Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based, by Gentry, Sahai and Waters